
ASSIGNMENT 29

Name: Om Prashant Kulawade

College: Zeal Polytechnic, Narhe, Pune

Branch: Artificial Intelligence and Machine Learning (AIML)

Domain: AIML (Artificial Intelligence and Machine Learning)

<https://github.com/omkulawade03/Frontend-Session-Assignments/tree/main/task29>

Import Required Libraries

```
import pandas as pd
import numpy as np
import joblib
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
```

[1] ✓ 2.9s

Q1. (Data Preparation)

Load the cleaned Emotions dataset (or the original train.txt after cleaning).

Separate the features (text) into X and target (emotion) into y.

Perform train-test split with test_size=0.20 and random_state=42.

Print the shape of X_train, X_test, y_train, and y_test.

Program Code:

#Questions 01

```

# =====
# Q1. (Data Preparation)
# =====
# Load the cleaned emotions dataset (or load train.txt and apply cleaning)
df = pd.read_csv('cleaned_emotions.csv')

# Handle any missing values if present in cleaned_text
df['cleaned_text'] = df['cleaned_text'].fillna('')

# Separate features (X) and target (y)
X = df['cleaned_text']
y = df['encoded_emotions']

# Perform train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

# Print the shapes
print("--- Q1. Train-Test Split Shapes ---")
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}\n")
[2] ✓ 0.1s
```

OUTPUT SCREENSHOT:

```
--- Q1. Train-Test Split Shapes ---  
X_train shape: (12800,)  
X_test shape: (3200,)  
y_train shape: (12800,)  
y_test shape: (3200,)
```

Q2. (Bag of Words – Basic)

Apply CountVectorizer (Bag of Words) on the training data:

Fit and transform X_train.

Transform X_test.

Print the shape of the resulting matrices.

Display the first 20 feature names (vocabulary words).

Program Code:

#Questions 02

```
# =====  
# Q2. (Bag of Words – Basic)  
# =====  
# Apply CountVectorizer for unigrams  
bow_vectorizer = CountVectorizer()  
X_train_bow = bow_vectorizer.fit_transform(X_train)  
X_test_bow = bow_vectorizer.transform(X_test)  
  
print("--- Q2. Bag of Words (Basic) ---")  
print(f"X_train_bow shape: {X_train_bow.shape}")  
print(f"X_test_bow shape: {X_test_bow.shape}")  
print("First 20 feature names:")  
print(bow_vectorizer.get_feature_names_out()[:20])  
print()
```

[5]

OUTPUT SCREENSHOT:

```
--- Q2. Bag of Words (Basic) ---  
X_train_bow shape: (12800, 13359)  
X_test_bow shape: (3200, 13359)  
First 20 feature names:  
['aa' 'aaaaand' 'aaaand' 'aac' 'aahhh' 'aaron' 'ab' 'abandon' 'abandoned'  
 'abandoning' 'abandonment' 'abated' 'abbigail' 'abc' 'abdomen'  
 'abdominal' 'abducted' 'abhorrent' 'abide' 'abilities']
```

Q3. (Bag of Words + MultinomialNB)

Train a Multinomial Naive Bayes model using the Bag of Words features:

Fit the model on X_train_bow and y_train.

Make predictions on the test set.

Calculate and print the accuracy score.

Program Code:

#Questions 03

```
▶ ▾  
# =====  
# Q3. (Bag of Words + MultinomialNB)  
# =====  
# Train Multinomial Naive Bayes model using BoW features  
nb_bow = MultinomialNB()  
nb_bow.fit(X_train_bow, y_train)  
  
# Make predictions and calculate accuracy  
y_pred_bow = nb_bow.predict(X_test_bow)  
accuracy_bow = accuracy_score(y_test, y_pred_bow)  
  
print("--- Q3. BoW + MultinomialNB ---")  
print(f"Bag of Words (Unigram) Test Accuracy: {accuracy_bow:.4f}\n")  
[8] ✓ 0.1s
```

OUTPUT SCREENSHOT:

```
--- Q3. BoW + MultinomialNB ---  
Bag of Words (Unigram) Test Accuracy: 0.7678
```

Q4. (Understanding Vocabulary)

Using the fitted CountVectorizer from Q2:

Print the total size of the vocabulary.

Show any 15 words from the vocabulary.

Convert one sample document from the training set into its BoW vector and display it.

Program Code:

#Questions 04

```
▶ ~  
# =====  
# Q4. (Understanding Vocabulary)  
# =====  
feature_names = bow_vectorizer.get_feature_names_out()  
  
print("--- Q4. Understanding Vocabulary ---")  
print(f"Total vocabulary size: {len(feature_names)}")  
print(f"Sample 15 words from vocabulary: {list(feature_names[100:115])}")  
  
# Convert one sample document into its BoW vector  
sample_idx = 0  
sample_text = X_train.iloc[sample_idx]  
sample_vector = X_train_bow[sample_idx].toarray()  
  
print(f"\nSample Document: '{sample_text}'")  
print(f"Corresponding BoW Vector (non-zero entries):")  
for idx, val in enumerate(sample_vector[0]):  
    if val > 0:  
        print(f"Word: '{feature_names[idx]}' -> Count: {val}")  
print()  
[9] ✓ 0.0s
```

OUTPUT SCREENSHOT:

```
--- Q4, Understanding Vocabulary ---
Total vocabulary size: 13359
Sample 15 words from vocabulary: ['aches', 'achieve', 'achieved', 'achievement', 'achievements', 'achieving', 'aching', 'achy', 'acin', 'acker', 'acknow']

Sample Document: 'refers course though cant help feeling somehow ironically retrospect loudons son kate mcgarrigle rather talented rufus wainwright'
Corresponding Bow Vector (non-zero entries):
Word: 'cant' -> Count: 1
Word: 'course' -> Count: 1
Word: 'feeling' -> Count: 1
Word: 'help' -> Count: 1
Word: 'ironically' -> Count: 1
Word: 'kate' -> Count: 1
Word: 'loudons' -> Count: 1
Word: 'mcgarrigle' -> Count: 1
Word: 'rather' -> Count: 1
Word: 'refers' -> Count: 1
Word: 'retrospect' -> Count: 1
Word: 'rufus' -> Count: 1
Word: 'somehow' -> Count: 1
Word: 'son' -> Count: 1
Word: 'talented' -> Count: 1
Word: 'though' -> Count: 1
Word: 'wainwright' -> Count: 1
```

Q5. (N-grams with Bag of Words)

Create a new CountVectorizer that uses bigrams (ngram_range=(1, 2)).

Fit and transform the training data.

Transform the test data.

Print the shape of the new feature matrix.

Display some of the bigram features.

Program Code:

#Questions 05

```
# =====  
# Q5. (N-grams with Bag of Words)  
# =====  
# Create CountVectorizer with unigrams and bigrams  
ngram_vectorizer = CountVectorizer(ngram_range=(1, 2))  
X_train_ngram = ngram_vectorizer.fit_transform(X_train)  
X_test_ngram = ngram_vectorizer.transform(X_test)  
  
print("--- Q5. N-grams with Bag of Words ---")  
print(f"X_train_ngram shape: {X_train_ngram.shape}")  
print(f"X_test_ngram shape: {X_test_ngram.shape}")  
print("Sample bigram/unigram features:")  
print(ngram_vectorizer.get_feature_names_out()[5000:5015])  
print()  
[30] ✓ 0.6s
```

OUTPUT SCREENSHOT:

```
--- Q5. N-grams with Bag of Words ---  
X_train_ngram shape: (12800, 91579)  
X_test_ngram shape: (3200, 91579)  
Sample bigram/unigram features:  
['audience therefore' 'audience wondering' 'audiences' 'audiences become'  
 'audiences go' 'audiobooks' 'audiobooks much' 'audre' 'audre lorde'  
 'auggie' 'auggie make' 'august' 'august attempt' 'aunt' 'aunt cant']
```

Q6. (N-grams + Model Training)

Train a MultinomialNB model using the bigram features created in Q5.

Calculate and print the accuracy on the test set.

Compare this accuracy with the unigram (basic BoW) accuracy from Q3.

Program Code:

#Questions 06

```
# --- Q6: (N-grams + Model Training) ---  
# Train MultinomialNB with n-gram features  
nb_ngram = MultinomialNB()  
nb_ngram.fit(X_train_ngram, y_train)  
  
y_pred_ngram = nb_ngram.predict(X_test_ngram)  
accuracy_ngram = accuracy_score(y_test, y_pred_ngram)  
  
print("--- Q6: N-grams + Model Training ---")  
print(f"Bag of Words (Unigram + Bigram) Test Accuracy: {accuracy_ngram:.4f}")  
print(f"Comparison: Unigram Accuracy = {accuracy_bow:.4f} vs Unigram+Bigram Accuracy = {accuracy_ngram:.4f}\n")  
[11] ✓ 0.0s
```

OUTPUT SCREENSHOT:

```
--- Q6. N-grams + Model Training ---  
Bag of Words (Unigram + Bigram) Test Accuracy: 0.7266  
Comparison: Unigram Accuracy = 0.7678 vs Unigram+Bigram Accuracy = 0.7266
```

Q7. (TF-IDF Vectorization)

Apply `TfidfVectorizer` on the same train-test split:

Fit and transform `X_train`.

Transform `X_test`.

Print the shape of the TF-IDF matrices.

Display the first 15 feature names.

Program Code:

#Questions 07



```
# =====  
# Q7. (TF-IDF Vectorization)  
# =====  
# Apply TfidfVectorizer  
tfidf_vectorizer = TfidfVectorizer()  
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)  
X_test_tfidf = tfidf_vectorizer.transform(X_test)  
  
print("--- Q7. TF-IDF Vectorization ---")  
print(f"X_train_tfidf shape: {X_train_tfidf.shape}")  
print(f"X_test_tfidf shape: {X_test_tfidf.shape}")  
print("First 15 TF-IDF feature names:")  
print(tfidf_vectorizer.get_feature_names_out()[:15])  
print()
```

[10]

OUTPUT SCREENSHOT:

```
--- Q7. TF-IDF Vectorization ---  
X_train_tfidf shape: (12800, 13359)  
X_test_tfidf shape: (3200, 13359)  
First 15 TF-IDF feature names:  
['aa' 'aaaaand' 'aaaand' 'aac' 'aahhh' 'aaron' 'ab' 'abandon' 'abandoned'  
 'abandoning' 'abandonment' 'abated' 'abbigail' 'abc' 'abdomen']
```

Q8. (TF-IDF + MultinomialNB)

Train a Multinomial Naive Bayes model using the TF-IDF features:

Fit the model.

Make predictions on the test set.

Calculate and print the accuracy score.

Program Code:

#Questions 08

```

# =====
# Q8. (TF-IDF + MultinomialNB)
# =====
# Train MultinomialNB with TF-IDF features
nb_tfidf = MultinomialNB()
nb_tfidf.fit(X_train_tfidf, y_train)

y_pred_tfidf = nb_tfidf.predict(X_test_tfidf)
accuracy_tfidf = accuracy_score(y_test, y_pred_tfidf)

print("--- Q8. TF-IDF + MultinomialNB ---")
print(f"TF-IDF Test Accuracy: {accuracy_tfidf:.4f}\n")

```

[15] ✓ 0.0s

OUTPUT SCREENSHOT:

```
--- Q8. TF-IDF + MultinomialNB ---  
TF-IDF Test Accuracy: 0.6609
```

Q9. (Comparison of Vectorizers)

Create a comparison table of the three approaches:

Bag of Words (Unigrams)

Bag of Words (Unigrams + Bigrams)

TF-IDF

Show the accuracy of each. Write a short observation about which method performed best and why.

Program Code:

#Questions 09

```
# Q9. (Comparison of Vectorizers)
# Create a comparison table of the three approaches:
# Bag of Words (Unigrams)
# Bag of Words (Unigrams + Bigrams)
# TF-IDF

comparison_data = {
    "Vectorization Approach": [
        "Bag of Words (Unigrams)",
        "Bag of Words (Unigrams + Bigrams)",
        "TF-IDF"
    ],
    "Test Accuracy": [
        accuracy_bow,
        accuracy_ngram,
        accuracy_tfidf
    ]
}

comparison_df = pd.DataFrame(comparison_data)
print("--- Q9. Comparison Table ---")
print(comparison_df.to_string(index=False))
print("\nObservation:")
print("TF-IDF or n-grams generally capture term importance or contextual word pairs better than basic BoW.")
print("TF-IDF penalizes frequent words that carry less semantic weight, often yielding robust performance for text classification tasks.\n")
```

OUTPUT SCREENSHOT:

```
--- Q9. Comparison Table ---
      Vectorization Approach  Test Accuracy
      Bag of Words (Unigrams)      0.767813
Bag of Words (Unigrams + Bigrams)  0.726562
                        TF-IDF      0.660937

Observation:
TF-IDF or N-grams generally capture term importance or contextual word pairs better than basic BoW.
TF-IDF penalizes frequent words that carry less emotional weight, often yielding robust performance for text classification tasks.
```

Q10. (Mini Project – Complete Vectorization Pipeline)

Using the Emotions dataset:

- 1. Load and clean the text (if not already cleaned).**
 - 2. Perform train-test split.**
 - 3. Apply both CountVectorizer and TfidfVectorizer.**
 - 4. Train MultinomialNB on both.**
 - 5. Compare their accuracy.**
 - 6. Save the best vectorizer and the best model using joblib.**
-

Program Code:

#Questions 10



```
# =====  
# Q10. (Mini Project - Complete Vectorization Pipeline)  
# =====  
print("--- Q10. Mini Project Pipeline Execution ---")  
# Best vectorizer and model selection based on accuracy comparison  
best_accuracy = max(accuracy_bow, accuracy_ngram, accuracy_tfidf)  
  
if best_accuracy == accuracy_tfidf:  
    best_vectorizer = tfidf_vectorizer  
    best_model = nb_tfidf  
    best_name = "TF-IDF + MultinomialNB"  
elif best_accuracy == accuracy_ngram:  
    best_vectorizer = ngram_vectorizer  
    best_model = nb_ngram  
    best_name = "Bag of Words (Bigrams) + MultinomialNB"  
else:  
    best_vectorizer = bow_vectorizer  
    best_model = nb_bow  
    best_name = "Bag of Words (Unigrams) + MultinomialNB"  
  
print(f"Best Performing Approach: {best_name} with Accuracy: {best_accuracy:.4f}")  
  
# Save the best vectorizer and best model using joblib  
joblib.dump(best_vectorizer, 'best_vectorizer.pkl')  
joblib.dump(best_model, 'best_model.pkl')  
print("Best vectorizer saved as 'best_vectorizer.pkl'")  
print("Best model saved as 'best_model.pkl'")
```

[12] ✓ 0.0s

OUTPUT SCREENSHOT:

```
--- Q10. Mini Project Pipeline Execution ---  
Best Performing Approach: Bag of Words (Unigrams) + MultinomialNB with Accuracy: 0.7678  
Best vectorizer saved as 'best_vectorizer.pkl'  
Best model saved as 'best_model.pkl'
```
